

# C Software Development

GNC Software Onboarding Lecture #1

```
// Don't worry about this for now, this line allows us to use printf
#include <stdio.h>

//main is the entry point of our program
int main(void) {
    //print "Hello, World!" and add a new line at the end
    printf("Hello, World!\n");
}
```

# Hello, World!

# Data Types

- ▶ Numeric
  - ▶ (unsigned/signed) int, char, long
  - ▶ float/double
- ▶ Arrays
  - ▶ Use brackets [N], where N is the size of the array
- ▶ Text
  - ▶ Char - Single character using the ASCII standard
  - ▶ Strings are represented as lists of chars with a special null ('\0') character at the end to indicate the string end
- ▶ Structs
  - ▶ A struct is a structured collection of the primary data types mentioned above
- ▶ Prepending a data type with the keyword 'const' makes it a constant

```
struct MyStruct {  
    float x;  
    int y[2];  
};  
  
int main(void) {  
    int a = 0;  
    unsigned char b = 'a';  
    const double c = 10.2;  
  
    char string[6] = {'H', 'e', 'l', 'l', 'o', '\0'};  
  
    struct MyStruct my_struct = {  
        .x = 10.0,  
        .y = {1, 2}  
    };  
}
```

```
int numbers[10];

for (int i = 0; i < 10; i++) {
    numbers[i] = i;
}
```

```
int number = 8;

if (number < 4) {
    //won't execute
} else if (number < 6) {
    //won't execute
} else {
    //will execute
}
```

```
int i;
while (i < 10) {
    numbers[i] = i;
    i++;
}
```

```
int number = 5;

if (number < 4) {
    //won't execute
} else if (number < 6) {
    //will execute
} else {
    //won't execute
}
```

# Control Flow

# Functions

- ▶ `<Return Type> <Function Name> (<Parameter 1>, <Parameter 2>, ...) {}`
- ▶ To call a function, use the function name with parameters in parenthesis

```
11  int add(int a, int b) {  
12      return a + b;  
13  }  
14  
15  void do_something(void) {  
16      //do something  
17  }
```

```
1  int main(void) {  
2      int sum = add(3, 4);  
3      do_something();  
4  }
```

# Function Prototypes

- ▶ The C compiler works top-to-bottom
  - ▶ If you define a function on line 7 and try to use it on line 4, the compiler won't know what function you are talking about
- ▶ Function prototypes solve this problem by defining the input parameters and return type of the function without actually defining the body of the function

```
1  int function_b(int x);  
2  
3  int function_a(int x) {  
4      return function_b(x);  
5  }  
6  
7  int function_b(int x) {  
8      return x + 1;  
9  }
```

# Pointers

- ▶ Denoted by a \* after the type name
- ▶ When you create a variable, think of it being stored on a “location” in your computer (called a memory address)
  - ▶ A **pointer** is a number that specifies where a value is stored
- ▶ Use the & symbol to create a pointer that points to a variable
- ▶ Use the \* symbol to access or modify the value the pointer points to
  - ▶ This is called **dereferencing**

```
#include <stdio.h>

int main(void) {
    int number = 5;
    int *number_ptr = &number;

    printf("number is: %d\n", number); // "number is 5"
    printf("number_ptr stores: %d\n", *number_ptr); // "number_ptr is 5"
    printf("number_ptr is: %p\n", number_ptr); // "number_ptr is: <memory address>"

    //A. We can either change the value stored in number_ptr
    *number_ptr = 6;

    printf("number is: %d\n", number); // "number is 6"
    printf("number_ptr stores: %d\n", *number_ptr); // "number_ptr stores 6"
    printf("number_ptr is: %p\n", number_ptr); // "number_ptr is: <memory address>"

    //B. Or we can change the value of number
    number = 10;

    printf("number is: %d\n", number); // "number is 10"
    printf("number_ptr stores: %d\n", *number_ptr); // "number_ptr stores 10"
    printf("number_ptr is: %p\n", number_ptr); // "number_ptr is: <memory address>"

    //Turns out A and B are doing the exact same thing
}
```

# Arrays as Pointers

- ▶ We can also use pointers to represent arrays in a different way
  - ▶ points to the start of the array
- ▶ Arrays are stored in contiguous memory locations, meaning that we can find the next element in the array by looking in the next memory address
- ▶ When we access elements of the array using square brackets, we are just adding an offset to that memory location
  - ▶ This is called **pointer arithmetic**

```
int main(void) {  
    int numbers[5] = {4, 6, 1, 2, 4};  
    int *numbers_ptr = numbers;  
  
    int first_num    = numbers_ptr[0];  
    first_num        = *numbers_ptr;  
  
    int second_num_1 = numbers_ptr[1];  
    second_num        = *(numbers_ptr + 1);  
}
```

```
int get_sum(int *numbers, int length) {  
    int sum = 0;  
  
    for (int i = 0; i < length; i++) {  
        sum += numbers[i];  
    }  
  
    return sum;  
}  
  
int main(void) {  
    int numbers[5] = {4, 6, 1, 2, 4};  
    int sum = get_sum(numbers, 5);  
}
```



# Pass by Value vs. Pass by Reference

- ▶ When you pass a parameter to a function, the data is copied from outside the function call to inside the function call
- ▶ This means that if we modify any parameters in the function itself, those modifications will not be reflected outside of the function

```
void plus_one(int num) {  
    num = num + 1;  
}  
  
int main(void) {  
    int num = 1;  
    plus_one(num);  
  
    //num will still be 1  
}
```

```
void plus_one(int *num) {  
    *num = *num + 1;  
}  
  
int main(void) {  
    int num = 1;  
    plus_one(&num);  
  
    //num will now be 2  
}
```

# The Danger of Pointers

```
#include <stdio.h>

int *dangeruou_function(int x) {
    int number = x;
    return &number;
}

int main(void) {
    int *ptr = dangeruou_function(5);
    dangeruou_function(7);

    printf("%d\n", *ptr); //ptr will now store the value 7 :0

    int *null_ptr = (int *) 0;
    printf("%d\n", *null_ptr); //segmentation fault

    int *random_ptr = (int *) 0xAFC12AB49;
    printf("%d\n", *random_ptr); //segmentation fault
}
```

- ▶ Pointers are a powerful tool, but can cause some serious headache if not treated carefully
- ▶ Some things you cannot do:
  - ▶ Return a pointer that references a local variable
  - ▶ Dereference a null pointer (memory address 0)
  - ▶ Dereference a pointer to memory your program does not own

# Memory Management

- ▶ In our previous examples, the compiler has known ahead of time how much memory we need
  - ▶ It can do this because we explicitly declared things such as array lengths
- ▶ ...but what if we don't know the size array we need at the time of writing the code?
- ▶ Enter malloc() and free()
  - ▶ malloc dynamically allocates memory and returns a pointer to the memory it has set aside
  - ▶ free does the opposite, returning the memory back to the operating system
- ▶ If you don't free your malloc'd memory, you have a memory leak!
  - ▶ This can cause your program to eventually run out of memory and crash
  - ▶ We almost always avoid using malloc in GNC for this reason

```
int array_size = //some non-constant value
int *array = (int*) malloc(array_size * sizeof(int));
//do something with array
free(array); //REMEMBER TO DO THIS!!
```

# Malloc/Free Example

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    //Get the length of the array. Don't worry about scanf
    int length = 0;
    printf("Enter the length of array: ");
    scanf("%d", &length);

    /* malloc takes the number of bytes to allocate as a parameter,
     * so we must multiply the number of ints by the size of an int in bytes */
    int *array = (int *) malloc(length * sizeof(int));

    for (int i = 0; i < length; i++) {
        //Get the next number. Don't worry about scanf
        int num = 0;
        printf("Enter number: ");
        scanf("%d", &num);

        array[i] = num;
    }

    //Print the numbers back to the user
    printf("The numbers you entered are: ");

    for (int i = 0; i < length; i++) {
        printf("%d ", array[i]);
    }

    printf("\n");

    //Return the memory back to the operating system
    free(array);
}
```

# The C Pre-Processor (Macros)

- ▶ Sometimes, you may want to define shortcuts for certain constants or for statements that are repeated often
- ▶ Macros are just a systematic way to tell the compiler to do text replacement
- ▶ Use the pre-processor directive `#define`
- ▶ Typically one-line
- ▶ Can have parameters just like functions, but not quite the same...
  - ▶ See `MULT_WRONG` vs `MULT`

```
1  #define SOME_CONSTANT 20
2  #define MULT_WRONG(a, b) (a * b)
3  #define MULT(a, b) ((a) * (b))
4
5  //i = 20
6  int i = SOME_CONSTANT;
7
8  //j = 1 + 2 * 3 = 7
9  int j = MULT_WRONG(1 + 2, 3);
10
11 //k = (1 + 2) * (3) = 9
12 int k = MULT(1 + 2, 3);
```

# Using Multiple Files

```
//lib.c

int add_one(int num) {
    return num + 1;
}
```

```
#include <stdio.h>

int add_one(int num);

int main(void) {
    printf("%d\n", add_one(2));
}
```

- ▶ Say we want to have a lib.c file that provides certain functions for main.c to use
- ▶ The compiler treats each .c file one at a time with no context shared between them
  - ▶ This means that we cannot just use a function from lib.c in main.c, because main.c does not define it
- ▶ We can use function prototypes to solve this
  - ▶ Although main.c does not provide an implementation for add\_one, it is enough for the compiler to know what the function prototype is
  - ▶ After the compiler runs, a separate program called the **linker** will then look for a definition for the function add\_one and link it to the function call in main.c

# The C Pre-Processor (Header Files)

- ▶ The previous example works fine, but what if we want to use more functions? What if we want to use lib.c in more than just main.c?
  - ▶ There would be a lot of repetition of function prototypes across multiple different files
- ▶ **Header files** allow us to consolidate multiple function prototypes in a separate file to then be “included” in any C file that wants to use those functions
- ▶ Use the `#include` keyword followed by the file name to tell the pre-processor to add the contents of the .h file to the start of the c file

```
//FILE - library.h
#define PARAM_VALUE 10

int my_library(int param);
```

```
//FILE - main.c
#include "library.h"

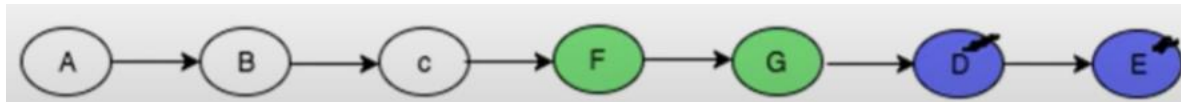
int main(void) {
    //value = 20
    int value = my_library(PARAM_VALUE);
}
```

```
//FILE - library.c
#include "library.h"

int my_library(int param) {
    return param * 2;
}
```

# Version Control (Git)

- ▶ In any software development team, members need a way to work on the same project simultaneously
- ▶ Changes you make in git are called **commits**, and these commits are structured in a **repository**, often hosted online on GitHub or elsewhere
  - ▶ A local copy of the repository exists on your computer, and may or may not be in sync with the online version
  - ▶ The online host is often referred to as the **origin** or **remote**
  - ▶ Syncing between the remote and local repositories is done through **pushes** and **pulls**
- ▶ At its most basic level, you can make a repository that has an entirely linear history of commits
  - ▶ One commit comes after the other and builds on the previous commit
- ▶ But if two developers are trying to work on the same code at the same time, we run into issues because they cannot continuously sync their changes

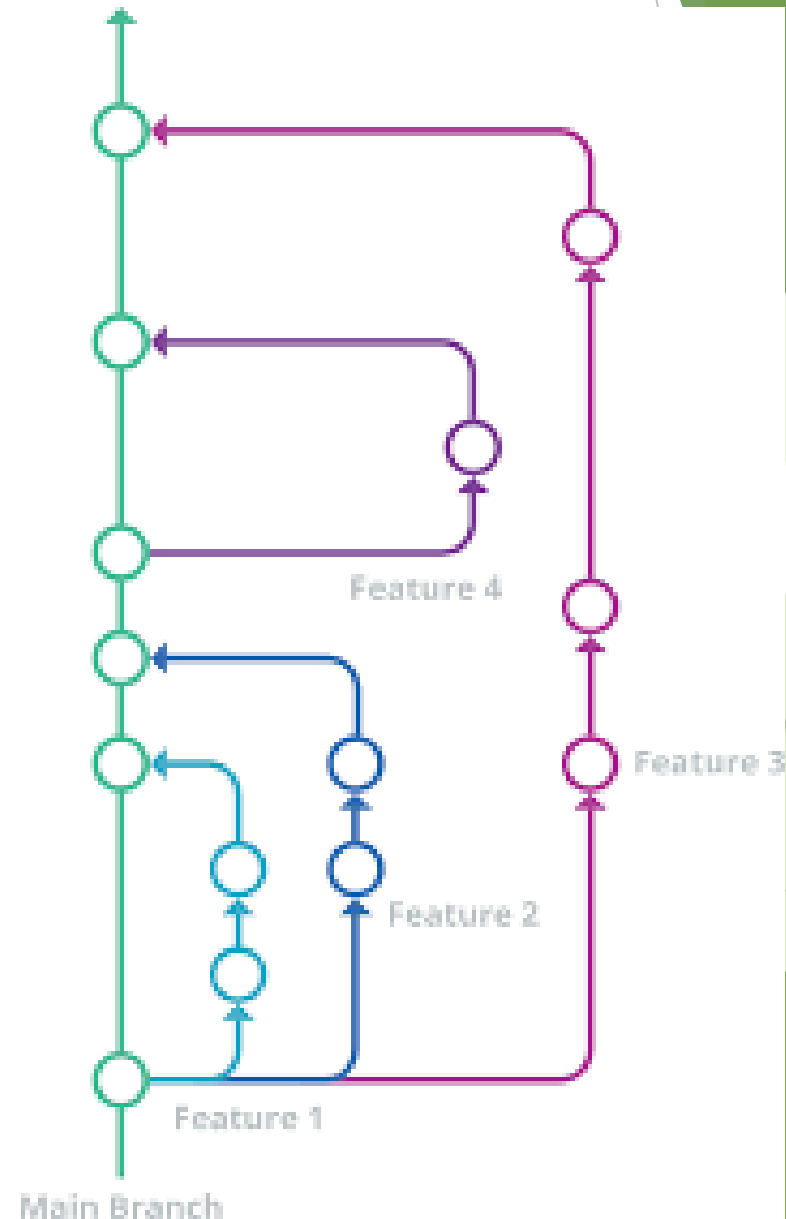


Example of a linear git history. Each color corresponds to a feature



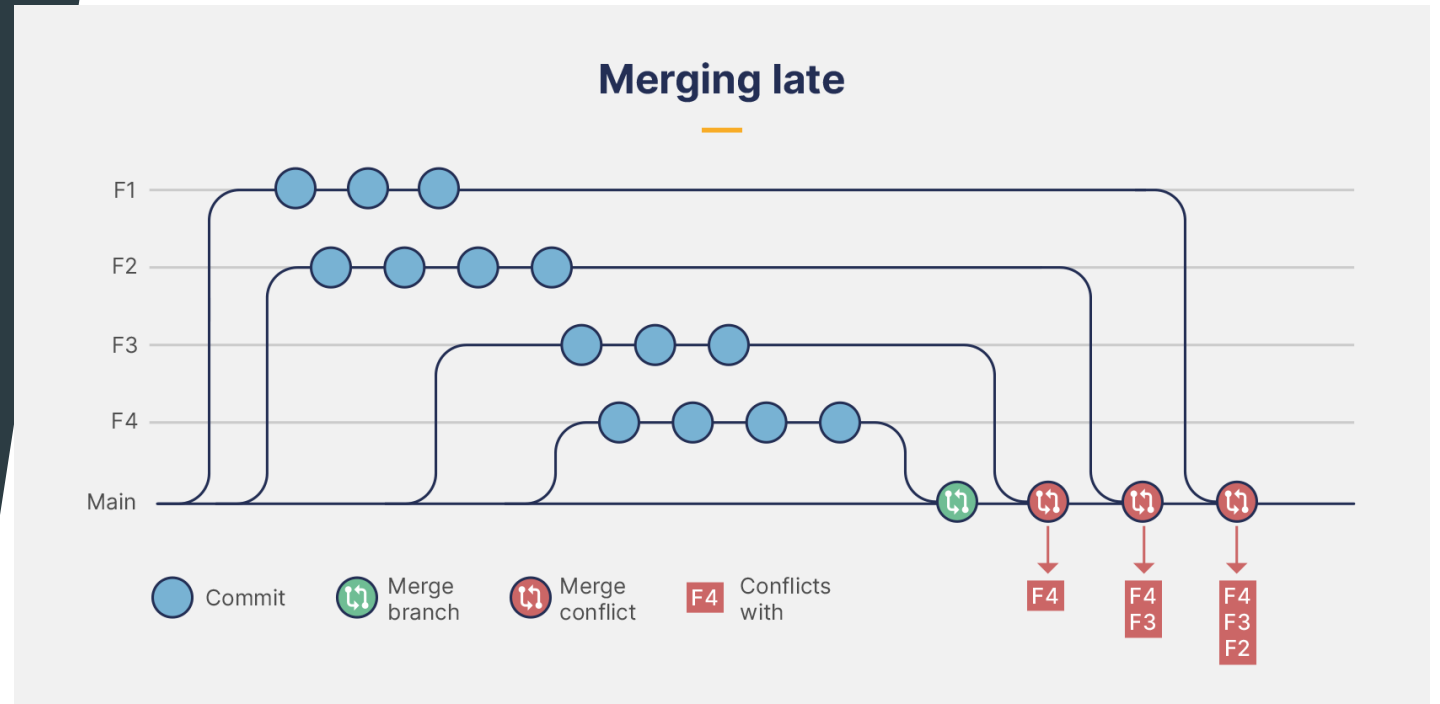
# Git Branches

- ▶ In git, a “branch” is an isolated line of development that starts from a single commit
  - ▶ Typically, a branch is created for each feature to be developed, or for each developer working on the project
  - ▶ Two developers typically can’t be working on the same branch at once
- ▶ Once development is complete on a branch, it is then “merged” with a mainline branch that contains the current state of the application



# Git Merges

- ▶ Imagine the following chain of events
  - ▶ Create feature branch A from main
  - ▶ Create feature branch B from main
  - ▶ Change file main.c in branch A
  - ▶ Make a different change to file main.c in branch B
  - ▶ Merge branch B into main
  - ▶ Merge branch A into main
- ▶ Which version of main.c ends up in main?
  - ▶ In all but the simplest cases, git produces a “merge conflict,” and a developer must manually decide which changes from branch A go into main.c and which changes from branch B go into main.c



# Basic Development Environment Setup

- ▶ If you are on windows
  - ▶ Install Windows Subsystem for Linux (WSL)
- ▶ Visual Studio Code
  - ▶ The happy blue one, not the evil purple one... it will take all of your memory and run with it
  - ▶ Extensions
    - ▶ C/C++
- ▶ Install git
  - ▶ Ubuntu/WSL: `sudo apt-get install git`
  - ▶ MacOS: `brew install git` (install brew from [brew.sh](https://brew.sh))
- ▶ Clone the flight software repository
  - ▶ `git clone https://github.gatech.edu/gnc/flight-software.git`
- ▶ For flight software development, there are quite a few other tools you need to install, which will be listed in the flight software repository. If not, talk to Sam



# Practice Exercises

- ▶ Go to <https://github.gatech.edu/gnc/software-onboarding> and do exercise 1
  - ▶ 1. Run “git clone <https://github.gatech.edu/gnc/software-onboarding>”
  - ▶ 2. Create a branch by running “git checkout -b <first name>-exercise1”
    - ▶ Ex. Sam would run “git checkout -b sam-exercise1”
    - ▶ This creates a new branch
  - ▶ 3. Do the exercise following the instructions in the exercise 1 README
  - ▶ 4. Stage your changes by running “git add .”
  - ▶ 5. Commit your changes by running “git commit -m <commit message>”
    - ▶ Ex. git commit -m “Sam exercise 1”
  - ▶ 6. Push your changes by running “git push origin <first name>-exercise1”
  - ▶ 7. Create a pull request in GitHub to merge your branch into the main branch

# Reference Materials

# Useful Links

- ▶ <https://learngitbranching.js.org> - Interactive git tutorial
- ▶ <https://www.learn-c.org/> - Interactive C tutorial

# Various C Utility Functions

- ▶ `memcpy(void* dest, const void* src, size_t num)`
  - ▶ Copies num bytes of src to dest
- ▶ `memset(void* ptr, int value, size_t num)`
  - ▶ Sets num bytes of ptr to value
- ▶ `realloc(void* ptr, size_t new_size)`
  - ▶ Reallocates a previously allocated pointer to a new size
- ▶ `calloc(size_t num, size_t size)`
  - ▶ Like malloc, but sets the values to 0
- ▶ `sprintf(char* str, const char* format, ...)`
  - ▶ Dynamically format a string using variables as parameters
- ▶ `strlen(const char* str)`
  - ▶ Get the length of a string by counting the number of characters until the null character ('\\0')
- ▶ `strcpy(char* dest, const char* src)`
  - ▶ Copies the string src to dest